Welcome to your next lesson: Ordering lists: sorted() vs .sort()

I'm getting your personalized lesson ready—using what you've learned, what's coming up, and the best way to help you build your skills.

Step by step, you're moving forward.

Imagine you have a list of game scores that came in this order:

Python

```
scores = [400, 1200, 800, 600]
```

Sometimes you want to reorder this list so it's from smallest to largest (or largest to smallest). In Python, there are two main ways to sort:

scores.sort() → changes the original list
sorted(scores) → makes a new sorted list, original stays the same

We'll go step by step into what that really means.

Quick check to start:

If you want to sort a list but also keep the original unchanged for later, which one do you think you'd want to use: the one that changes the original, or the one that makes a new list?

sorted(scores) the one that makes a new list

Exactly right 🎯 — sorted(scores) is the one that makes a new list and keeps scores unchanged.

Let's look at both side by side with a tiny example.

Think of two actions:

In-place sort with .sort()

You tell Python: "Rearrange these items right here."
The original list is changed.
Copy-and-sort with sorted()

You tell Python: "Give me a new sorted version, but don't touch my original."

Visually:

Python

```
numbers = [3, 1, 2]

numbers.sort()       # in-place
# numbers is now [1, 2, 3]
```

Python

```
numbers = [3, 1, 2]

new_list = sorted(numbers)  # copy
```

```
# numbers is still [3, 1, 2]
# new_list is [1, 2, 3]
```

Before we touch the code editor:

Does this difference — “changes the original” vs “returns a new list” — make sense so far, or should I re-explain it with another real-world example?

Makes sense so far

What do you think happens here? 👇

You have this code:

```
numbers = [5, 2, 9]
result = numbers.sort()
```

After this runs, which statement is true?

A.

numbers is now sorted, and result is the new sorted list.

B.

numbers is unchanged, and result is the new sorted list.

C.

numbers is now sorted, and result is None.

Nice work, Ron 🎉 You picked the exact right answer.

In:

Python

```
numbers = [5, 2, 9]
result = numbers.sort()
.sort() changes numbers in place, so after it runs, numbers becomes [2, 5, 9].
.sort() does not return the sorted list — it returns None, so result is None.
```

So: “numbers is now sorted, and result is None” is correct.

Next step: I want to show you how sorted() behaves differently, then we’ll use key= and reverse= to get custom order. Ready to look at sorted() on a simple example next?

Yes, I'm ready

Awesome 😄

Let’s look at sorted() very clearly, side by side with .sort().

Start with this list:

Python

```
scores = [400, 1200, 800, 600]
```

Using .sort() (in-place):

Python

```
scores = [400, 1200, 800, 600]
scores.sort()

# now:
# scores -> [400, 600, 800, 1200]
```

Using sorted() (makes a new list):

Python

```
scores = [400, 1200, 800, 600]
sorted_scores = sorted(scores)

# now:
# scores         -> [400, 1200, 800, 600]   (unchanged)
# sorted_scores  -> [400, 600, 800, 1200]   (new list)
```

So the key idea:

.sort() → changes the list itself, returns None
sorted() → does not change the original, returns a new sorted list
Make sense so far, or want me to walk through a real-life example (like a "high scores" board) to reinforce it?

It makes sense

Great 👍

Now let's add one more idea: custom order.

So far, sorting has meant:

numbers → smallest to largest
strings → A to Z

But what if you have:

Python

```
words = ["python", "is", "fun"]
```

You can sort them by length instead of alphabetically using sorted() with a key:

Python

```
def by_length(word):
    return len(word)

words = ["python", "is", "fun"]
result = sorted(words, key=by_length)

# result will be: ["is", "fun", "python"]
```

Step by step:

Python calls by_length on each word → gets lengths [6, 2, 3]
It sorts by those lengths → order becomes shortest to longest
Does this idea of "use a function with key= to decide how to sort" make sense so far?

Yes, that makes sense

What do you think? Take your time with the question below 👇

You have this list:

```
scores = [10, 50, 30]
```

You want a sorted copy but also want to keep scores unchanged. Which line does that?

A.

```
scores.sort()
```

B.

```
sorted_scores = scores.sort()
```

C.

sorted_scores = sorted(scores)

Nice job again, Ron 🎯 You picked exactly the right line:

Python

sorted_scores = sorted(scores)
Why it’s correct:

sorted(scores) → makes a new sorted list and leaves scores unchanged.
That new list is stored in sorted_scores, so now you have:
scores → original [10, 50, 30]
sorted_scores → [10, 30, 50]

The other two:

scores.sort() → changes scores itself.
sorted_scores = scores.sort() → sorts scores in place and then puts None into sorted_scores.
Next, I’d like to show you how to reverse the order using reverse=True (for example, biggest score first). Want to try that next?

Yes, let's try it

Great 😊

We’ll go one small step: reverse order.

Normally:

Python

scores = [400, 1200, 800, 600]
sorted_scores = sorted(scores)
[400, 600, 800, 1200]
If you want largest to smallest, you add reverse=True:

Python

scores = [400, 1200, 800, 600]
sorted_scores = sorted(scores, reverse=True)
[1200, 800, 600, 400]
So:

reverse=False (default) → smallest → largest
reverse=True → largest → smallest
Can you explain in your own words what reverse=True is telling Python to do when sorting? Just a short sentence is fine.

it tells python to sort in the opposit direction from the default of lowest to highest thus it sorts largest to lowest

Exactly 💯 — that’s a perfect explanation.

reverse=True = “sort, but flip the direction from the default,”

so numbers go highest → lowest instead of lowest → highest.

Now I want to show you one more key idea for this lesson: stable sorting — where items that “tie” keep their original order.

After that, we'll do one final quick question to lock in .sort() vs sorted().

Ready for the stability idea?

Yes, I'm ready

Cool, let's go step by step 🧱

Think of a to□do list with priorities, where some tasks have the same priority:

Python

```
tasks = [
    ("email boss", 2),
    ("pay bills", 1),
    ("take a walk", 2),
]
```

Imagine we sort by the priority number (1 = most important).

Both "email boss" and "take a walk" have priority 2.

A stable sort means:

If two items have the same sort key, they keep the same order relative to each other as in the original list.

So after sorting by priority, "email boss" will still come before "take a walk" because it came first originally.

Python's sorted() and .sort() are stable — they keep that original order for "ties".

Does that idea of "ties keep their original order" feel clear, or should I show a tiny concrete example with actual Python code next?

Can you show an example?

Absolutely, let's make it concrete 👇

Start with this list of (name, score) where some scores are the same:

Python

```
players = [
    ("Alice", 10),
    ("Bob", 20),
    ("Carol", 20),
    ("Dave", 10),
]
```

We sort by the score (the second item in each pair):

Python

```
def by_score(player):
    return player[1]   # the score

sorted_players = sorted(players, key=by_score)
```

Logically, the scores after sorting are: 10, 10, 20, 20.

Because Python's sort is stable, the result will be:

Python

```
[
    ("Alice", 10),   # 10, came before Dave's 10 originally
    ("Dave", 10),    # 10
    ("Bob", 20),     # 20, came before Carol's 20 originally
    ("Carol", 20),   # 20
]
```

So:

The two 10 scores keep their order: Alice then Dave
The two 20 scores keep their order: Bob then Carol

That's stability: ties keep their original order.

Does this example make the stability idea click for you?

Yes, it makes sense

What do you think? Take your time with the question below 👇

You have this code:

```
numbers = [3, 1, 2]
result = sorted(numbers, reverse=True)
```

After it runs, which statement is true?

A.

numbers is now [3, 2, 1] and result is None.

B.

numbers is still [3, 1, 2] and result is [3, 2, 1].

C.

numbers is now [1, 2, 3] and result is [3, 2, 1].

Nice, Ron — you nailed it again 🎯

For:

Python

```
numbers = [3, 1, 2]
result = sorted(numbers, reverse=True)
```

sorted(numbers, reverse=True) creates a new list in reverse order → [3, 2, 1]
The original numbers is unchanged → still [3, 1, 2]
That new reversed-sorted list is stored in result

So the true statement is:

numbers is still [3, 1, 2] and result is [3, 2, 1].

You've now seen:

.sort() → in-place, returns None
sorted() → new list, original unchanged
reverse=True → flips the sort order
Sorts are stable → ties keep original order

Let’s lock this in with one final check about .sort() vs sorted(). What do you think?

sure

What do you think? Pick the one that keeps nums the same 👇

Which of these leaves the original list unchanged after it runs?

A.

```
nums = [4, 1, 3]
nums.sort()
```

B.

```
nums = [4, 1, 3]
result = nums.sort()
```

C.

```
nums = [4, 1, 3]
result = sorted(nums)
```

Perfect choice again, Ron 🎉

This code:

Python

```
nums = [4, 1, 3]
result = sorted(nums)
```

is the only one that:

uses sorted(nums) → creates a new sorted list
leaves nums unchanged as [4, 1, 3]
puts the sorted copy ([1, 3, 4]) into result

The other two both call .sort(), which always changes the original list in place.

You’ve hit all the key ideas in this lesson:

.sort() → sorts in place, returns None
sorted() → returns a new sorted list, original stays the same
key= → controls how items are compared
reverse=True → flips the order
Python’s sort is stable → ties keep their original order

You’ve been answering everything correctly — nicely done 🔥

Are you ready to wrap up this lesson and move on to the next one, or do you want to practice with a small coding exercise on sorting first?

I'm ready to move on

Awesome work on this lesson, Ron 🎓🔥

Here’s what you now know how to do:

Use .sort() to sort a list in place (and know it returns None).
Use sorted() to get a new sorted list while keeping the original unchanged.
Use reverse=True to flip the sort order (like highest score first).
Use key= with a simple function (like one using len) to control how things are sorted.
Explain that Python’s sort is stable: items with the same key keep their original order.
You answered everything correctly — that shows real understanding, not just guessing ☑

As you go forward, anytime you need to sort, ask yourself:

“Do I want to change this list?” → use .sort()

"Do I want a separate sorted copy?" → use sorted(...)
You're ready for the next lesson. When you're set, click into the next one in Learn, and I'll be there to keep going with you 🚀

How would you rate this lesson?

Message Maestro...